

AI Job Hunt Tracker

ACM CODEHUB PROJECT-BASED FELLOWSHIP

Shahan Ayub

SP26-BSE-010 | COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS

Project Report

1. Methodology

Database Schema & Status History

For this project, I used **MongoDB** because it works well with **Node.js** and **Express**. Every job application is stored as one document containing information such as:

- Company name
- Job title
- Job description
- Job URL
- Industry
- Role type
- Notes
- Follow-up date
- AI match score
- Strengths
- Missing skills
- Suggestions

Instead of only storing the current application status, I also created a **statusHistory** array.

Whenever the user changes the application status (for example, from **Applied** to **Interview**), the backend automatically adds a new record with the updated status and the current date instead of replacing the previous one.

This allows the application to keep track of the complete history of every job application and makes it easier to monitor progress over time.

Gmail API Integration

To send follow-up emails, I used the **Google Gmail API** with **OAuth 2.0** authentication instead of using a normal SMTP password.

The user first signs in with Google. After successful authentication, Google provides access and refresh tokens which **are stored securely in MongoDB**.

Whenever the user sends a follow-up email, the backend retrieves the saved OAuth tokens, formats the email in the required format (Base64URL), and sends it using the Gmail API. Using OAuth is more secure because passwords are never stored inside the application.

2. EDA Insights

Before writing any scraper, I created a **Jupyter Notebook** to inspect the HTML structure of different job websites.

The goal was to find reliable HTML elements that could be used to extract:

- Company name
- Job title
- Job description

Google Careers

Google Careers was the easiest website to scrape.

Although many CSS classes looked random, the page structure stayed mostly consistent. I selected stable elements and successfully extracted all required information.

LinkedIn

LinkedIn pages were more complicated.

Some selectors changed between pages, so I inspected multiple job posts before choosing more reliable HTML elements. After updating the selectors, the scraper successfully extracted the company, title and description. **`a[href*="/company/"]`** was commonly used for company names.

Wellfound

Wellfound protected its pages using **DataDome Bot Protection**.

Whenever Playwright tried opening a job page, the request was redirected to a CAPTCHA page instead of the actual job listing. Since the HTML containing the job details never loaded, Playwright could not scrape the information.

Indeed

Indeed also blocks automated browsers.

Instead of loading the job page, it displayed a security page and prevented Playwright from accessing the content. Because of this restriction, automatic scraping could not be completed.

Deployment Decision (A setback)

While testing locally, I initially used **Playwright** because it can render JavaScript-heavy websites.

However, during deployment on **Render**, Playwright required downloading large Chromium binaries and OS-level dependencies that frequently cause build failures on cloud servers that were difficult to support as Render's free tier has strict memory limits and cannot handle Playwright's heavy headless browser installation..

For websites that could be scraped without rendering JavaScript, I switched to **Axios** and **Cheerio** because they are much lighter, faster and easier to deploy.

For websites protected against bots (such as Wellfound and Indeed), I added a **manual text input option** where users can paste the job description, company name and job title directly. This ensures that the AI analysis still works even if automatic scraping fails.

3. Challenges Faced

1. Playwright Blocking

The biggest challenge was that some websites blocked Playwright completely.

Google Careers and LinkedIn worked after adjusting selectors, but Wellfound and Indeed redirected every automated request to security or CAPTCHA pages.

Instead of trying to bypass these protections, I implemented a **manual job description textbox**. Users can simply paste the job description themselves, allowing the AI to analyze it without relying on the scraper.

2. PDF Resume Text

When extracting text from uploaded PDF resumes, the output contained many unnecessary line breaks, spaces and formatting issues.

This caused problems when sending the text to the Gemini API.

To solve this, I cleaned the extracted text using regular expressions before sending it for analysis. This made the AI responses much more reliable.

3. Secure OAuth Token Storage

At first, I considered storing Gmail authentication information temporarily, but that would be lost whenever the server restarted.

Instead, I stored the OAuth tokens securely in MongoDB.

Whenever the user wants to send a follow-up email, the backend retrieves these saved tokens. If the user has not authenticated yet, the backend returns an authorization error and asks the user to sign in first.

This approach keeps the application secure while also making the login process convenient for the user.